

§ 3.3 列表

3.3.1 头、尾节点

List对象的内部组成及逻辑结构如图3.1所示，其中私有的头节点（header）和尾节点（trailer）始终存在，但对外并不可见。对外部可见的数据节点如果存在，则其中的第一个和最后一个节点分别称作首节点（first node）和末节点（last node）。

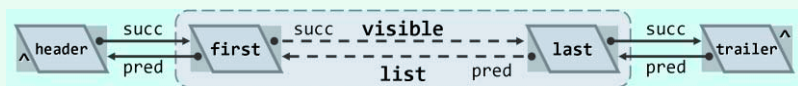


图3.1 首（末）节点是头（尾）节点的直接后继（前驱）

就内部结构而言，头节点紧邻于首节点之前，尾节点紧邻于末节点之后。这类经封装之后从外部不可见的节点，称作哨兵节点（sentinel node）。由代码3.2中List::valid()关于合法节点位置的判别准则可见，此处的两个哨兵节点从外部被等效地视作NULL。

设置哨兵节点之后，对于从外部可见的任一节点而言，其前驱和后继在列表内部都必然存在，故可简化算法的描述与实现。比如，在代码3.2中为实现first()和last()操作，只需直接返回header->succ或trailer->pred。此外更重要地，哨兵节点的引入，也使得相关算法不必再对各种边界退化情况做专门的处理，从而避免出错的可能，我们稍后将对此有更实际的体会。

尽管哨兵节点也需占用一定的空间，但只不过是常数规模，其成本远远低于由此带来的便利。

3.3.2 默认构造方法

创建List对象时，默认构造方法将调用如代码3.3所示的统一初始化过程init()，在列表内部创建一对头、尾哨兵节点，并适当地设置其前驱、后继指针构成一个双向链表。

```
1 template <typename T> void List<T>::init() { //列表初始化，在创建列表对象时统一调用
2     header = new ListNode<T>; //创建头哨兵节点
3     trailer = new ListNode<T>; //创建尾哨兵节点
4     header->succ = trailer; header->pred = NULL;
5     trailer->pred = header; trailer->succ = NULL;
6     _size = 0; //记录规模
7 }
```

代码3.3 列表类内部方法init()

如图3.2所示，该链表对外的有效部分初始为空，哨兵节点对外不可见，此后引入的新节点都将陆续插入于这一对哨兵节点之间。



图3.2 刚创建的List对象

在列表的其它构造方法中，内部变量的初始化过程与此相同，因此都可统一调用init()过程。该过程仅涉及常数次基本操作，共需运行常数时间。

3.3.3 由秩到位置的转换

鉴于偶尔可能需要通过秩来指定列表节点，可通过重载操作符“[]”，提供一个转换接口。

```
1 template <typename T> //重载下标操作符，以通过秩直接访问列表节点（虽方便，效率低，需谨慎）
2 T& List<T>::operator[](int r) const { //assert: 0 <= r < size
3     ListNodePosi(T) p = first(); //从首节点出发
4     while (0 < r--) p = p->succ; //顺数第r个节点即是
5     return p->data; //目标节点，返回其中所存元素
6 }
```

代码3.4 重载列表类的下标操作符

具体地如代码3.4所示，为将任意指定的秩 r 转换为列表中对应的元素，可从首节点出发，顺着后继指针前进 r 步。只要秩 r 合法，该算法的正确性即一目了然。其中每步迭代仅需常数时间，故该算法的总体运行时间应为 $O(r + 1)$ ，线性正比于目标节点的秩。

相对于向量同类接口的 $O(1)$ 复杂度，列表的这一效率十分低下——其根源在于，列表元素的存储和访问方式已与向量截然不同。诚然，当 r 大于 $n/2$ 时，从`trailer`出发沿`pred`指针逆行查找，可以在一定程度上减少迭代次数，但就总体的平均效率而言，这一改进并无实质意义。

3.3.4 查找

■ 实现

在代码3.2中，列表ADT针对整体和区间查找，重载了操作接口`find(e)`和`find(e, p, n)`。其中，前者作为特例，可以直接调用后者。因此，只需如代码3.5所示，实现后一接口。

```
1 template <typename T> //在无序列表内节点p（可能是trailer）的n个（真）前驱中，找到等于e的最后者
2 ListNodePosi(T) List<T>::find(T const& e, int n, ListNodePosi(T) p) const { //0<=n<=rank(p)<_size
3     while (0 < n--) //对于p的最近的n个前驱，从右向左
4         if (e == (p = p->pred)->data) return p; //逐个比对，直至命中或范围越界
5     return NULL; //p越出左边界意味着区间内不含e，查找失败
6 } //失败时，返回NULL
```

代码3.5 无序列表元素查找接口`find()`

■ 复杂度

以上算法的思路及过程，与无序向量的顺序查找算法`Vector::find()`（代码2.10）相仿，故时间复杂度也应是 $O(n)$ ，线性正比于查找区间的宽度。

3.3.5 插入

■ 接口

为将节点插至列表，可视具体要求的不同，在代码3.6所提供的多种接口中灵活选用。

```
1 template <typename T> ListNodePosi(T) List<T>::insertAsFirst(T const& e)
2 { _size++; return header->insertAsSucc(e); } //e当作首节点插入
3
```

```

4 template <typename T> ListNodePosi(T) List<T>::insertAsLast(T const& e)
5 { _size++; return trailer->insertAsPred(e); } //e当作末节点插入
6
7 template <typename T> ListNodePosi(T) List<T>::insertBefore(ListNodePosi(T) p, T const& e)
8 { _size++; return p->insertAsPred(e); } //e当作p的前驱插入
9
10 template <typename T> ListNodePosi(T) List<T>::insertAfter(ListNodePosi(T) p, T const& e)
11 { _size++; return p->insertAsSucc(e); } //e当作p的后继插入

```

代码3.6 列表节点插入接口

可见，这些接口的实现，都可转化为列表节点对象的前插入或后插入接口。

■ 前插入

将新元素 e 作为当前节点的前驱插至列表的过程，可描述和实现如代码3.7所示。

```

1 template <typename T> //将e紧靠当前节点之前插入于当前节点所属列表（设有哨兵头节点header）
2 ListNodePosi(T) ListNode<T>::insertAsPred(T const& e) {
3     ListNodePosi(T) x = new ListNode(e, pred, this); //创建新节点
4     pred->succ = x; pred = x; //设置正向链接
5     return x; //返回新节点的位置
6 }

```

代码3.7 ListNode::insertAsPred()算法

图3.3给出了整个操作的具体过程。插入新节点之前，列表局部的当前节点及其前驱如图(a)所示。该算法首先如图(b)所示创建新节点 new ，构造函数同时将其数据项置为 e ，并令其后继链接 $succ$ 指向当前节点，令其前驱链接 $pred$ 指向当前节点的前驱节点。随后如图(c)所示，使 new 成为当前节点前驱节点的后继，使 new 成为当前节点的前驱（次序不能颠倒）。最终如图(d)所示，经过如此调整，新节点即被顺利地插至列表的这一局部。

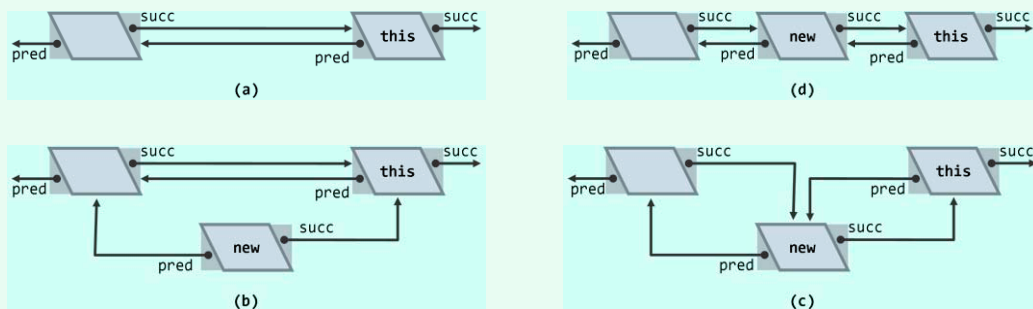


图3.3 ListNode::insertAsPred()算法

请注意，列表规模记录的更新由代码3.6中的上层调用者负责。另外，得益于头哨兵节点的存在，即便当前节点为列表的首节点，其前驱也如图(a)所示必然存在，故不必另做特殊处理。当然，在当前节点即首节点时，前插入接口等效于List::insertAsFirst()。

■ 后插入

将新元素 e 作为当前节点的后继插至列表的过程，可描述和实现如代码3.8所示。

```
1 template <typename T> //将e紧随当前节点之后插入于当前节点所属列表 (设有哨兵尾节点trailer)
2 ListNodePosi(T) ListNode<T>::insertAsSucc(T const& e) {
3     ListNodePosi(T) x = new ListNode(e, this, succ); //创建新节点
4     succ->pred = x; succ = x; //设置逆向链接
5     return x; //返回新节点的位置
6 }
```

代码3.8 `ListNode::insertAsSucc()`算法

后插入的操作过程以及最终效果与前插入完全对称，不再赘述。

■ 复杂度

上述两种插入操作过程，仅涉及局部的两个原有节点和一个新节点，且不含任何迭代或递归。若假设当前节点已经定位，不计入此前的查找所消耗的时间，则它们都可在常数时间内完成。

3.3.6 基于复制的构造

与向量一样，列表的内部结构也是动态创建的，故利用默认的构造方法并不能真正地完成新列表的复制创建。为此，需要专门编写相应的构造方法，通过复制某一已有列表来构造新列表。

■ `copyNodes()`

尽管这里提供了多种形式，以允许对原列表的整体或局部复制，但其实质过程均大同小异，都可概括和转化为如代码3.9所示的底层内部方法`copyNodes()`。在输入参数合法的前提下，`copyNodes()`首先调用`init()`方法，创建头、尾哨兵节点并做相应的初始化处理，然后自 p 所指节点起，从原列表中取出 n 个相邻的节点，并逐一作为末节点插至新列表中。

```
1 template <typename T> //列表内部方法：复制列表中自位置p起的n项
2 void List<T>::copyNodes(ListNodePosi(T) p, int n) { //p合法，且至少有n-1个真后继节点
3     init(); //创建头、尾哨兵节点并做初始化
4     while(n--) { insertAsLast(p->data); p = p->succ; } //将起自p的n项依次作为末节点插入
5 }
```

代码3.9 列表类内部方法`copyNodes()`

根据此前的分析，`init()`操作以及各步迭代中的插入操作均只需常数时间，故`copyNodes()`过程总体的运行时间应为 $O(n + 1)$ ，线性正比于待复制列表区间的长度 n 。

■ 基于复制的构造

如代码3.10所示，基于上述`copyNodes()`方法可以实现多种接口，通过复制已有列表的区间或整体，构造出新列表。其中，为了复制列表 L 中自秩 r 起的 n 个相邻节点，`List(L, r, n)`需借助重载后的下标操作符，找到待复制区间起始节点的位置，然后再以此节点作为参数调用`copyNodes()`。根据3.3.3节的分析结论，需要花费 $O(r + 1)$ 的时间才能将 r 转换为起始节点的位置，故该复制接口的总体复杂度应为 $O(r + n + 1)$ ，线性正比于被复制节点的最高秩。由此也可再次看出，在诸如列表之类采用动态存储策略的结构中，循序访问远非有效的方式。